Engineering Reboot 2026

Day 19 — JSON Fundamentals

Today's objective

Learn to persist Python objects using JSON and understand why JSON has become the standard data interchange format for modern software.

By the end of today you should be comfortable with:

• importing the json module

• serializing Python objects (json.dump)

• deserializing JSON (json.load)

• working with nested dictionaries and lists

• updating JSON data

• writing modified data back to disk

Exercise 1 — Creating JSON

Program: json_write_demo.py

Create a Python dictionary representing a person.

```
person = {
    "name": "Glenn",
    "age": 53,
    "city": "Newark",
    "skills": [
        "Python",
        "Git",
        "Docker"
    ]
}
```

Write it to **person.json** using **json.dump()**.

Exercise 2 — Reading JSON

Program: **json_read_demo.py**. Read **person.json** using json.load() and display Name, Age, City and Skills. Practice accessing nested lists.

Exercise 3 — Pretty Printing

Modify Exercise 1 to use **indent=4**. Compare person.json before and after indentation and understand why pretty-printed JSON is useful.

Exercise 4 — Updating JSON

Load the JSON, add another skill (e.g. VS Code), write the modified JSON back, and verify that the file changed.

Mini Project

student_manager.py with **students.json**.

```
{
    "id": 1001,
    "name": "...",
    "course": "...",
    "gpa": ...
}
```

• Display all students

• Search student

• Add student

• Save changes

Stretch Goal

```
{
    "students": [
        ...
    ]
}
```

Compare which structure feels easier to maintain.

Challenge

Create **engineering_profile.json** that represents you as a software engineer. Design the JSON schema yourself.

```
{
    "name": "...",
    "experience": {
        "software": 22,
        "construction": 2
    },
    "languages": [],
    "tools": [],
    "goals": [],
    "projects": []
}
```